

字符串常量池

1. 创建对象的思考

下面两种创建String对象的方式相同吗？

```
public static void main(String[] args) {  
    String s1 = "hello";  
    String s2 = "hello";  
    String s3 = new String("hello");  
    String s4 = new String("hello");  
    System.out.println(s1 == s2); // true  
    System.out.println(s1 == s3); // false  
    System.out.println(s3 == s4); // false  
}
```

上述程序创建方式类似，为什么s1和s2引用的是同一个对象，而s3和s4不是呢？

在Java程序中，类似于：1， 2， 3， 3.14，“hello”等字面类型的常量经常频繁使用，**为了使程序的运行速度更快、更节省内存**，Java为8种基本数据类型和String类都提供了常量池。

"池" 是编程中的一种常见的, 重要的**提升效率**的方式, 我们会在未来的学习中遇到各种 "内存池", "线程池", "数据库连接池"

比如：家里给大家打生活费的方式

1. 家里经济拮据，每月定时打生活费，有时可能会晚，最差情况下可能需要向家里张口要，速度慢
2. 家里有矿，一次性打一年的生活费放到银行卡中，自己随用随取，速度非常快

方式2，就是池化技术的一种示例，钱放在卡上，随用随取，效率非常高。常见的池化技术比如：数据库连接池、线程池等。

为了节省存储空间以及程序的运行效率，Java中引入了：

1. **Class文件常量池**：每个Java源文件编译后生成Class文件中会保存当前类中的字面常量以及符号信息
2. **运行时常量池**：在Class文件被加载时，.Class文件中的常量池被加载到内存中称为运行时常量池，运行时常量池每个类都有一份
3. **字符串常量池**

此处简单了解下，后序在讲JVM时会给同学们详细阐释。

2. 字符串常量池(StringTable)

字符串常量池在JVM中是StringTable类，实际是一个固定大小的HashTable(一种高效用来进行查找的数据结构，后序给大家详细介绍)，不同JDK版本下字符串常量池的位置以及默认大小是不同的：

JDK版本	字符串常量池位置	大小设置
Java6	(方法区)永久代	固定大小：1009
Java7	堆中	可设置，没有大小限制，默认大小：60013
Java8	堆中	可设置，有范围限制，最小是1009

关于方法区、堆等内存结构的具体局部，后序JVM中会给大家详细介绍。

3. 再谈String对象创建

由于不同JDK版本对字符串常量池的处理方式不同，此处Java8 HotSpot上分析

1. 直接使用字符串常量进行赋值

```
public static void main(String[] args) {
    String s1 = "hello";
    String s2 = "hello";
    System.out.println(s1 == s2); // true
}
```

```
public static void main(String[] args) {
    String s1 = "hello";
    String s2 = "hello";
    String s3 = new String("world");
    String s4 = new String("world");
}
```

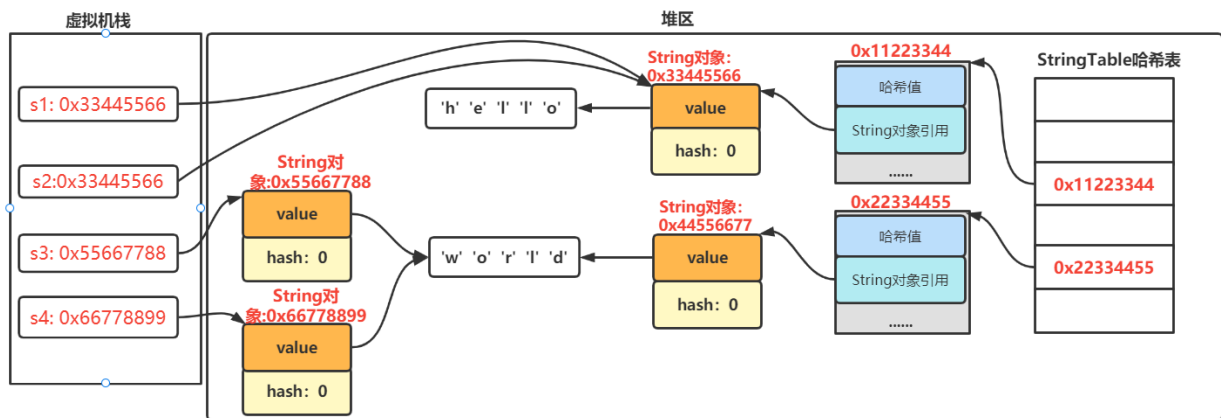
```
public String(@NotNull String original) {
    this.value = original.value;
    this.hash = original.hash;
}
```

Bytecode viewer showing instructions for the String constructor:

- 1: ldc #2 <hello>
- 2: astore_1
- 3: ldc #2 <hello>
- 4: astore_2
- 5: new #3 <java/lang/String> 1
- 6: dup 2
- 7: ldc #4 <world> 3
- 8: invokespecial #5 <java/lang/String.<init>> 4
- 9: astore_3 5
- 10: new #3 <java/lang/String>
- 11: dup
- 12: ldc #4 <world>
- 13: invokespecial #5 <java/lang/String.<init>>
- 14: astore_4
- 15: return

- ① new: 在堆上开辟String对象大小的空间，并将对象中成员初始化成0，然后将空间首地址压栈
- ② dup: 将栈顶元素，即String对象空间的首地址拷贝一份到栈顶备用
- ③ ldc: 将常量池中"world"对象引用拷贝到栈顶
- ④ invokespecial: 调用String类构造方法，此时：要取走栈顶2个元素
即："world"对象的引用 以及 第一步new的空间String空间的地址
- ⑤ astore2: 用栈顶元素给s2赋值

2. 通过new创建String类对象



结论：只要是new的对象，都是唯一的。

通过上面例子可以看出：使用常量串创建String类型对象的效率更高，而且更节省空间。用户也可以将创建的字符串对象通过 `intern` 方式添加进字符串常量池中。

3. intern方法

`intern` 是一个native方法(Native方法指：底层使用C++实现的，看不到其实现的源代码)，该方法的作用是手动将创建的String对象添加到常量池中。

```
public static void main(String[] args) {
    char[] ch = new char[]{'a', 'b', 'c'};
    String s1 = new String(ch); // s1对象并不在常量池中
    //s1.intern();           // s1.intern(); 调用之后，会将s1对象的引用放入到常量池中
    String s2 = "abc";       // "abc" 在常量池中存在了，s2创建时直接用常量池中"abc"的引用
    System.out.println(s1 == s2);
}

// 输出false
// 将上述方法打开之后，就会输出true
```

注意：在Java6 和 Java7、8中Intern的实现会有些许的差别